INDEX

Name: G. Selvakumar Std: Div: Roll no: Sub:

S. No.	Date	Title	Page No.	Teacher's Sign. / Remarks
1	20/1/25	Study of various measuring instruments used in Physics practical		
2	27/1/25	Study of different types of resistors & potentiometers		
3	3/2/25	Ohm's law		
4	10/2/25	Series and parallel circuits and Kirchhoff's laws		
5	17/2/25	Joule's law		
6	24/2/25	Time Period and Capacitive reactance		
7	3/3/25	Verify the characteristics of Diode and Zener diode		
8	10/3/25	Full wave and Bridge rectifier		
9	17/3/25	Study of Cathode ray oscilloscope		

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
10	29/9/25	Implementation of secure Authentication in client Server chat application		
11	6/10/25	Implementing with Packet with client server chat application		
12	13/10/25	Reading of network logs		
13	13/10/25	Implementation of Ping Program		
14	12/10/25	Packet sniffing using Raw sockets		

Done 9/10/25

20/7/26

Study of various Network Command used in Linux And windows

Basic networking commands

1. arp -a: ARP is short form of address resolution protocol it will show ip address of your computer along with ip address & MAC address of router

2. hostname: this is the simplest of all TCP / IP commands it simply displays the name of your computer

3. ip config/all: this command displays detailed configuration information about your connection

4. nbstat -a: this command helps in solving problems with NetBios name resolution

5. netstat: [Network Statistics] It displays a variety of statistics about computers active TCP / IP connection

it is a command line tool for monitoring network connection both incoming and outgoing as well as viewing routing tables, interface statistics etc. Eg: netstat -r

6. nslookup: Tool used for performing DNS lookups in Linux. it is used to display DNS details

NS lookup can operate in two modes: interactive & non interactive

Eg: nslookup www.google.com

7. pathping : pathping is unique to windows and is basically a combination of the ping and tracert commands.

Pathing traces the route to the destination address then launches a 25 sec test of each router along the way, gathering statistics on the rate of data loss along each hop

8. ping : [packet Internet Groper] . best way to test connectivity between two nodes

ICMP [Internet control message protocol] to communicate to other devices

ping hostname (ping localhost)
ping ip address (ping 4.2.2.2)
ping fully qualified domain name
(ping www.facebook.com)

9. Route : this command is used to show to IP routing table. It is primarily used to setup static routes to specific host or network via interface.

Some Important Linux networking commands.

1.

ip <options> <objects> <command>

(a) [root@server~] # ip address show

show IP address assigned to an interface

(b) [root@server~] # ip address add 192.168.1.254/24 dev enp03

assign an IP to an interface

(c) [root@server~] # ip address del 192.168.1.254/24 dev enp03

To delete an ip on an address

(d) [root@server~] # ip link set eth0 up

Alter the status of interface by bringing the interface eth0

(f) eth0: [root@server~] # ip link set eth0 promise on

Alter the status of interface by enabling promiscuous mode

(g) eth0: [root@server~] # ip route add default via 192.168.1.254 dev eth0

(h) Add a route to 192.168.1.0/24 via gateway at 192.168.1.254

[root@server~] # ip route add 192.168.1.0/24 via 192.168.1.254

(i) eth0: [root@server] # ip route add 192.168.1.0/24 dev eth0

Add route that can be reached on device.

(j) [root@server ~] # ip route delete 192.168.1.0/24
via 192.168.1.254
Delete the route 192.168.1.0/24 via gateway
192.168.1.254

(k) [root@server ~] # ip route get 10.10.1.4
Display the route taken for 10.10.1.4

2. ifconfig:
tool for configuring and troubleshooting networks

3. mtr:
mtr <options> hostname/ip

(a) [root@server ~] # mtr google.com
Basic mtr command shows you the statistics, including each hop with time & loss %.

(b) [root@server ~] # mtr -g google.com
show numeric ip address

(c) [root@server ~] # mtr -b google.com
show numeric ip address & host names

(d) [root@server ~] # mtr -c 10 google.com
set no. of pings that you want

4. tcpdump: designed for capturing & displaying packets

(a) [root@server ~] # dnf install -y tcpdump

(b) [root@server ~] # tcpdump -D

(c) [root@server ~] # tcpdump -i eth0 -c 10
capture traffic to & from the host

(d) [root@server ~] # tcpdump -i eth0 src host
8.8.8.8
for traffic coming from 8.8.8.8

(e) [root@server~] # tcpdump -i eth0 dst host 8.8.8.8
for outbound traffic going to 8.8.8.8

(f) [root@server~] # tcpdump -i eth0 net 10.1.0.0/24
capture traffic to and from a network

(g) [root@server~] # tcpdump -i eth0 port 53
capture traffic to and from port number.

5. ping:

Ping is network utility used to check connectivity between two devices on a network. It sends ICMP Echo Request messages to a destination host and waits for ECHO Reply messages

eg:

ping google.com

Ping 10 times:

ping -c 10 google.com

* Reply Received: connection is working properly
* Destination host unreachable: No route to the destination host
* Request time out: No Response Received from the host
* unknown host: Hostname is Incorrect or cannot be Resolved

A good ping result should have low latency and 0% paket loss.

Result:

Hence the various networking command used in Linux and windows are studied and Executed. Successfully.

Practical - 2

28/4/26

Aim: Study of different types of Network cables

a) understand different types of network cable

different types of cables in network cable

1. unshielded Twisted pair (UTP) cable
2. Shielded Twisted pair (STP) cable
3. Coaxial cable
4. Fibre optic cable

cable Type	category	Maximum Data Transmission	Adv / disadvantages	Application / use
UTP	category 3	10bps	Advantages: * Cheaper in cost * Easy to install as they have a smaller overall diameter Disadvantages: * more prone to (EMI) Electromagnetic interface and noise	10 Base - T Ethernet
	category 5	upto 100 mbps		Fast Ethernet gigabit Ethernet
	category 5e	10Gbps		Fast Ethernet gigabit Ethernet
STP	category 6, 6a	10Gbps	Advantages: * Shielded * Faster than UTP * Less susceptible to noise and interface Disadvantages: * Expensive * Greater installation effort	Gigabit Ethernet 10Gb Ethernet (55m) widely used in data centres
SSTP	category 7	10Gbps		Gig. bit Ethernet (100m)

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantage: *high bandwidth * Immune to interface * low loss bandwidth * versatile Disadvantage: * limited distance * cost * size inflexibility	speed of signal is 500m Television network high speed internal connections
fiber optics cable	single mode multimode	100G bps or	Advantage: *high speed * high bandwidth * high security * long distance Disadvantage: * Expensive * Requires skilled installers	* maximum distance of fibre optics cable is around 100 meters.

Result:

Thus the different types of Network cables are studied successfully.

4/8/22

14/8/26

Cisco Packet Tracer

Aim:

To study cisco packet Tracer tool installation and user interface overview

Introduction:

A simulator, as the name suggests; simulate network devices and its environment Packet Tracer is an existing network design, simulation and modelling tool.

* It allows you to model complex systems without the need for dedicated equipment
* It helps you to practice your network configuration and trouble shooting skills via computer or an android or iOS based mobile device
* It is available for both the unix and windows desktop environments.
* Protocols in packet tracer are coded to work and behave in the same way as they would on real Hardware.

Analyze the behaviour of network devices using cisco packet tracer simulator.

(i) peer to peer

(ii) Switch

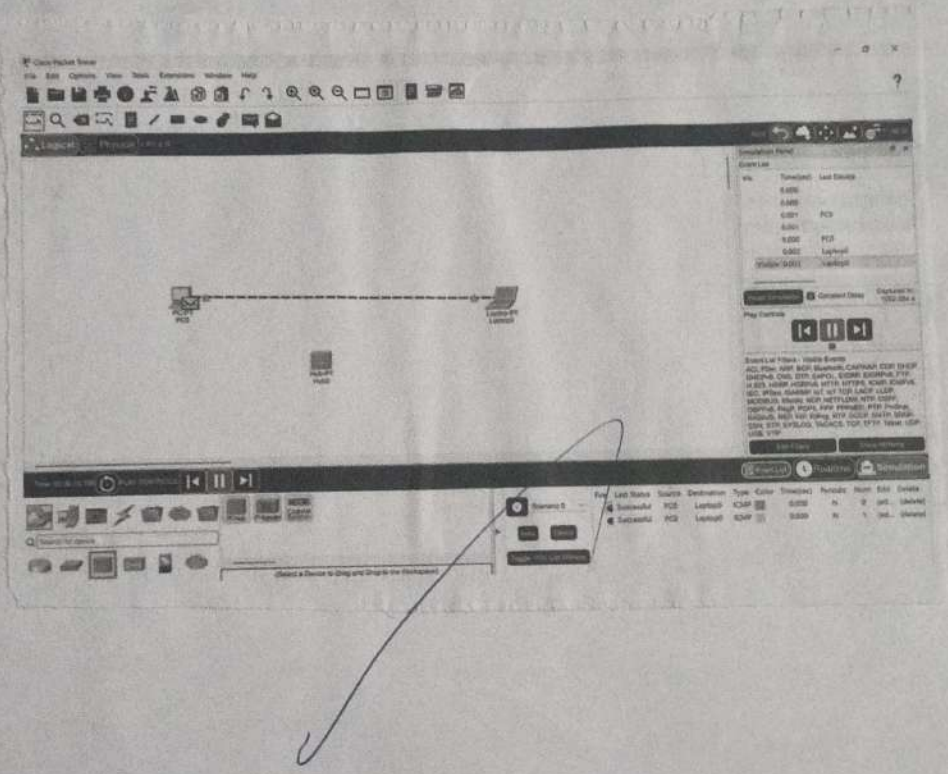
(iii) Hub

(i) peer to peer.

A peer to peer network is a network in which computer communicate directly with each other without Requiring a dedicated central server

Ex:

Two pc's connected directly using a cable and communicating with each other.



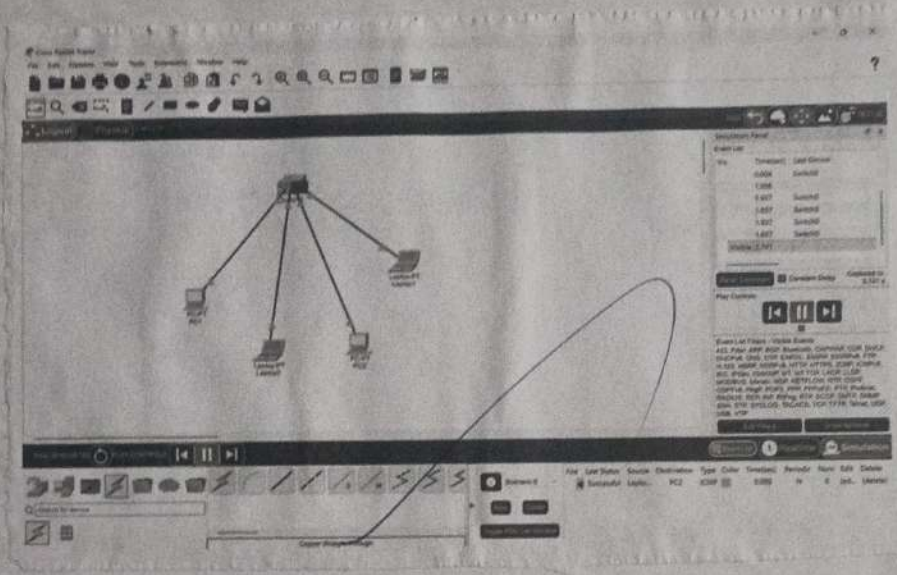
(ii) Switch:

A switch is a networking device used to connect multiple devices on a network and forward data only to the intended destination device.

Ex:

PC1 → Switch → PC3

Switch knows where PC3 is and forwards the frame only toward PC3. Switch only works at the Data Link Layer (Layer 2) of the OSI model it uses mac address. Sends data to the Required host

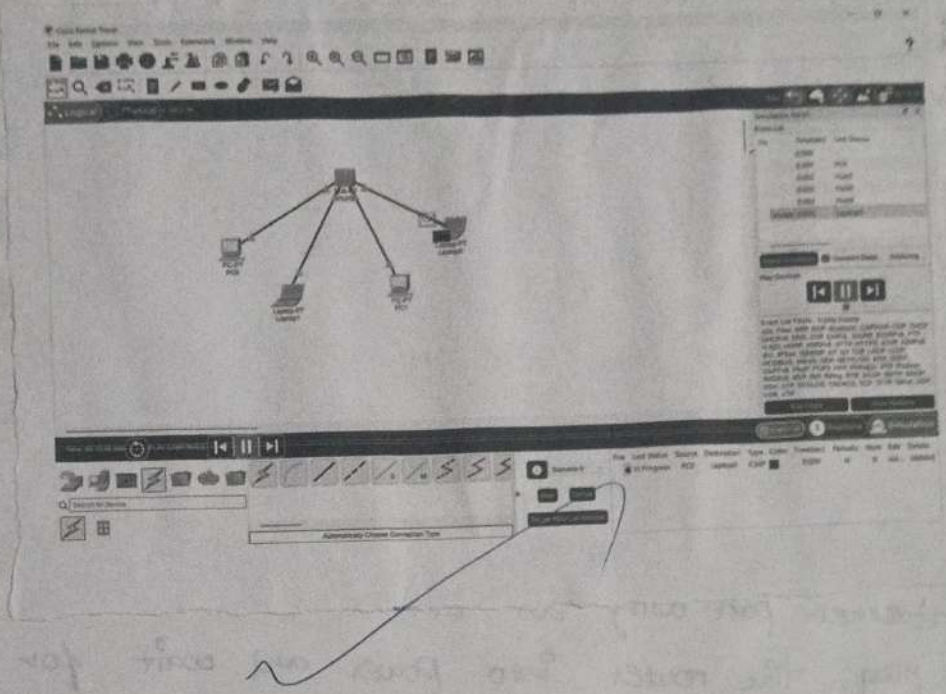


(iii) Hub.

A Hub is a basic networking device used to connect multiple computers in a network. It receives data from one devices and broadcast it to all connected devices. Hub works mainly at physical Layer (Layer 1) of the OSI Model

Ex:

If PC1 sends data to PC3 through a Hub, the Hub sends that data to PC2, PC3, PC4 etc.



Result:

Hence, the Cisco packet tracer tool and its user Interface has been studied.

18/8/26

10/8/26 Exp 4 Setup and configure a LAN using switch and Ethernet cables.

What is LAN?

LAN Stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a home, school, or office building

How to set up LAN.

Step 1: Connect the Hardware
→ Place your router in a central location near a power outlet
→ connect your modem to the router's WAN or internet port using an Ethernet cable.
→ plug the router into power and wait for it to turn on.

Step 2: connect your devices.
→ wired setup: plug one end of an Ethernet cable into your computers network port and the other end into one of the numbered LAN ports on the router.

Step 3: Configure the router.
→ Type your IP address (192.168.1.1 or 192.168.0.1) into the address bar and press enter.
→ login using the default username and password printed on the router.

Step 4: check the connectivity between switch and other machine. by using ping command in the command prompt of the device.

10/8/26

Steps: Try to access the shared folder from other computers of the networks.

Result:
wireless LAN using switch and ethernet cables are executed successfully in cisco packet-tracer.

10/8/26

18/8/26

EXP: 5 WIRE SHARK

Aim:

Experiment on packet capture ~~tool~~ wireshark

To filter, capture, view, packets in wireshark tool. capture 100 packets from the Ethernet IEEE 802.3 LAN interface and save it

Procedure:

* Select Local Area connection in wireshark
* To Capture Option
* Select Stop capture automatically after 100 Packets
* Then click, start capture
* Search TCP packets in search bar
* To see flow graph click statistics flow graph
* Save the packets

1. Create a filter to display only TCP/UDP Packets, Inspect the packets and provide the flow graph.

* Select LAN connection in wireshark
* To capture option
* Select Stop capture automatically after 100 packets
* then click start capture
* Save the packets

2. Create a filter to display only ARP packets and Inspect the packets procedure.

* Go to capture option
* Select Stop capture automatically after 100 packets
* Then click Start capture
* Search ARP packets in search bar
* Save the packets.

3. Create a filter to display only HTTP packets and inspect the packets procedure.

* Select Local Area connection in wireshark
* Go to capture option
* Select Stop capture automatically after 100 packets.
* Then click save capture
* Search HTTP packets in search bar.

4. Create a file to display only DNS packets and provide the flow graph

Procedure:

* Go to capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search DNS packets in search bar
* To see the flow graph click statistics flow graph

5. Create a filter to display only DHCP packets and inspect the packets.

* Select Local Area connection in wireshark
* Go to captured option
* Select Stop capture automatically after 100 packets
* Then click start capature
* Search DHCP packets in search bar
* Save the packets.

Results.

Wireshark on packet capture tool experiment are learned and executed successfully.

18/8/25

-1/9/25 Exp6

Aim:

detec
conc
&tra

send

*

*

*

4/9/26 Exp6 Error detection & correction using Hamming code.

Aim:

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

Sender:

* Input to sender file should be a text of any length. program should be convert the text to binary.
* Apply hamming code concept on the binary data and add redundant bits to it.
* Save this output in a file called channel.

code:

```
data = input ("Enter 4 bits.")
d1, d2, d3, d4 = map (int, data)
p1 = d1^d2^d4
p2 = d1^d3^d4
p3 = d2^d3^d4
code = [p1, p2, d1, p4, d2, d3, d4]
print ("Sender Hamming code:", "".join
(map (str, code)))
```

Receiver side:

* Receive the hamming code from the sender
* Enter the received code.
* calculate the parity check values.
* find the error position using the parity values.
* correct the error by changing the bit
* Display the corrected hamming code.

Code:

```
r= list(map(int, input("Enter received 7bits :")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]
error = r1*1 + r2*2 + r4*4
if error == 0:
    print("No Error")
else:
    print("Error at position :", error)
    r[error-1] ^= 1
print("Corrected code :", "".join(map(str, received)))
```

Input:

Enter 4 data bits : 1011
Sender hamming code : 0110011
Received 7bits : 0b100 11

Output:

Error at position : 2

Corrected code : 0110011

22/9/26

Exp.7 NMAP to Discover Live Hosts using Nmap scans on the Tryhackme platform

Aim:

To Discover live hosts using NMAP scans (ARP, ICMP, TCP(UDP) on the tryhackme platform Room. Link: https://tryhackme.com/room/nmapol.

Procedure:

Task1 :- Introduction:

* ARP Scan: this scan uses ARP requests to discover live hosts
* ICMP scan: this scan uses ICMP requests to identify live hosts.
* TCP/UDP ping scan: this scan sends packets to TCP ports and UDP ports to determine like hosts

Task2 : Subnetworks:

Network segments - It is a collection of computer linked together through a common "medium".

Subnetwork - It is usually the equivalent of one or more networks segment either together and configured to utilize the same router in an IP networks.

* Subnets with /16
* Subnets with /24

Task 3: Enumerating targets:

We need to identify the targets we want to scan before we conduct any "action"

1) List
- <Machine -IP> <Scanme.nmap.org> <example.com>
- will scan 3 IP addresses

2) range
- 10.11.12.15-20
- will scan 6 IP addresses

nmap - sL <TARGETS>

Task 4 :- Discovering Live Hosts.

1) ARP - from Link layer - commonly used for man-in-middle-Attack (ARP Spoofing and ARP poising)

2) ICMP from network layer - commonly used in DDOS attacks

3) Tcp from Transport layer - SYN flooding, Tcp reset, Tcp session hijacking attacks.

4) UDP from Transport layer - Also used in DDOS attacks.

Task 5: Nmap Host Discovery using ARP.

1) when a privileged user tries to scan LAN (Local Area Network), Nmap uses ARP requests.

2) to discover: internet hosts without port scanning - like systems are nmap -sn targets

3) nmap -PR -sn <TARGETS>
-PR specifies that you only want an ARP scan.

4) nmap - PR - Sn <MACHINE IP> 124

5) nmap - PR - Sn < ATTACH BOX IP> /24

Task 6 : Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP echo requests to discover live host

2) Nmap - PE - Sn TARGET to check host is live

3) -PP option instructs Nmap to use ICMP timestamp requests

4) nmap - PP - Sn TARGET

4) -PM option can enable this scan to ICMP address mask requests

nmap - PM - SN TARGET

Task 7 : Nmap host discovery using TCP and UDP

1) Nmap to use TCP SYN ping use - PS followed by the port number, range, list or a combination of this option

-PS21, -PS25 targets ports 21, 22, 23, 24, 25.

2) nmap - PS - Sn TARGET check the TCP port is open

3) TCP ACK ping - you must be running nmap as a privileged user.

4) nmap - PA - Sn TARGET to discover the online hosts on the target's subnet.

5) UDP ping - sending a UDP packet to an open port is unlikely to result in a response

6) If we transmit a UDP packet to a closed UDP port we should expect to receive an ICMP port unreachable packet -

nmap - PU - SN - TARGET

Task 8: - using Reverse - DNS Lookup

1) Nmap is to use reverse - DNS Internet hosts

2) -R option to query the DNS Server even for offline hosts

Task 9: Summary:

ARP Scan, ICMP Echo Scan, ICMP Timestamp Scan, ICMP address mask Scan, TCP SYN ping Scan TCP ACK, ping Scan, UDP ping Scan.

Result:

NMAP to discover live host using tryhackme platform was successfully executed.

22/1/2

28/9/26

Exp: 8 End to End communication at transport Layer

a)

Aim: implement echo unit server using TCP/UDP Packets

Procedure:

* create a socket using python socket programming
* Specify the IP address and port number.
* Bind the server socket to the IP address and port
* make the server listen for client connections
* Connect the client to the server
* Send and receive message between client and server
* Display the received message
* close the socket connection

Code: TCP - server.

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind("127.0.0.1", 5000)
server.listen(1)
print("server is waiting for connection...")
conn, addr = server.accept()
print("client connected")
while True:
data = conn.recv(1024).decode()
if not data:
    break
```

```
print ("client :", data)
conn.send(data.encode())
conn.close()
server.close()
```

Tcp - Receiver (client)

```
import socket
client = socket.socket(socket.AF_INET,
        socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message:")
client.send(message.encode())
reply = client.recv(1024).decode()
print ("server:", reply)
client.close()
```

Terminal 1 - Server

```
python tcp_server.py
Server is waiting for
connection...
client connected
client: Hello
```

Terminal 2 - client

```
python tcp_client.py
Enter message: Hello
server: Hello
Tcp echo is working
```

Result:

The Tcp/UDP packets using echo client server socket program is executed succesfully.

28/9

b)

Aim:

implement chat client server. using TCP Packets.

code:

chat - server side

```
import socket
server = socket.socket(socket.AF_INET, socket.
        SOCKET_STREAM)
Server.bind(("127.0.0.1", 6000))
Server.listen(1)
print("waiting for client...")
conn, addr = server.accept()
print("client connected")
while True:
    message = conn.recv(1024).decode()
    if message.lower() == "bye":
    print("client ended chat")
    break
    print("client:", message)
    reply = input("server:")
    conn.send(reply.encode())
    if reply.lower() == "bye":
conn.close()
Server.close()
```

chat - client side.

```
import socket
client = socket.socket(socket.AF_INET: socket.
        SOCK_STREAM)
client.connect(("127.0.0.1", 6000))
print("connected to server")
```

```
while True:
        message = input("Client : ")
        client.send(message.encode())
        if (message.lower() == "bye":
            break
        reply = client.recv(1024).decode()
        print("Server: "_ reply)
        if reply.lower() == 'bye':
        break
client.close()
```

Terminal 1

```
Python chat_server.py
waiting for client ...
client connected
client : Hello
Server : Hi
client : How are you?
Server : fine.
```

Terminal 2

```
Python chat_client.py
connected to server
client : Hello
Server : Hi
client : How are you?
Server : Fine
```

Result:

The Chat Programming using sockets By Tcp client & server was implemented successfully.

28/9/24

28/9/24 Exp 7: Sliding window Protocol

Aim:

Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow from one node to another.

code:

```
import time
window = int(input("Enter window size : "))
message = input("Enter message :")
start = 0
while start < len(message)
print("SENDER")
end = start + window
if end > len(message):
    end = len(message)
#Sender buffer
send_buffer = []
for i in range(start, end):
    frame = (i, message[i])
    sender_buffer.append(frame)
    print("Frame No:", i, "DATA :", message[i])
print("\n Sender Buffer :")
print(sender_buffer)

#Delay
print("\n waiting for receiver ....")
time.sleep()
# Receiver
print("RECEIER")
expected = start
error = False
for frame in sender_buffer:
```

```
frame_no = frame[0]
data = frame[1]
print("Received Frame:", frame_no, "DATA:", data)
if frame_no == expected:
    expected = expected+1
else:
    error = True
    break.
# Receiver Buffer
if error:
    receiver_buffer = ("NACK", expected)
    print("In Receiver Buffer:")
    print(receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("ACK", ack)
    print("In Receiver Buffer:")
    print("receiver_buffer")
# sender reads ACK | NACK
print("In SENDER CHECKING RESPONSE")
response = receiver_buffer[0]
number = receiver_buffer[1]
if response == "ACK":
        print("ACK received:", number)
        start = number+1
else:
    print("NACK received:") number
    start = number.
print("TRANSMISSION COMPLETED")
```

output:

Enter window size: 3

Enter message : HELLO

SENDER

Frame no: 0 DATA: H

Frame no: 1 DATA: E

Frame no: 2 DATA: L

Sender Buffer:

[(0,'H'),(1,'E'),(2,'L')] waiting for receiver ....

waiting for receiver ....

RECEIVER:

Received Frame: 0 DATA: H

Received Frame: 1 DATA: E

Received Frame: 2 DATA: L

Receiver Buffer:

('ACK', 2)

SENDER CHECKING RESPONSE

ACK Received: 2

SENDER:

Frame no: 3 DATA: L

Frame no: 4 DATA: O

RECEIVER:

Received frame: 3 DATA: L

Received frame: 4 DATA: O

Receiver Buffer:

('ACK', 4)

SENDER CHECKING RESPONSE

ACK Received: 4

TRANSMISSION COMPLETED.

Result:

Sliding window protocols are learned and executed successfully.

28/9/24

Exp-10 i

29/9/24 P

Aim:

To

and re

Subnetting

network

it helps

reduces

network

Algorithm

* Star

* pl

Pa

*

*

*

*

*

*

*

*

*

Exp-10 implementation of subnetting in cisco packet tracer simulator
29/9/24

Aim:

To implement subnetting using cisco packet tracer and verifying communication between the subnet-

Subnetting:

It is a process of dividing large network into smaller logical network called subnets it helps in efficient utilization of IP address, reduces unnecessary network traffic & improve network management

Algorithm:

* Start cisco packet tracer
* place one router, two switches & four PCs on the work space
* connect 2PC to switch & both switches to the router
* Assign static IP address & subnet mask to all PCs
* configure IP address on the router interfaces connected to the two subnets
* configure default gateway for each PC
* check connectivity between PCs with same subnet using ping command
* Test communication between PCs belonging to different subnets
* Disconnect (or) disable the route interface & repeat communication test
* observe & record result

Procedure:

Step 1: Design Network Topology

* connect PC0 & PC1 to Switch0
* connect PC2 & PC3 to switch1

Step 2: Assign IP address

Device	Ipaddress	subnetmask	Default- Gateway
PC0	192.168.1.2	255.255.255.0	192.168.1.1
PC1	192.168.1.3	255.255.255.0	192.168.1.1
PC2	192.168.1.4	255.255.255.0	192.168.1.1
PC3	192.168.1.5	255.255.255.0	192.168.1.1

Step 3: Configure the Router.

* open CLI
* Terminal

Enable
configuration terminal

interface gigabit ethernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit

interface gigabit Ethernet 0/1
ip address 192.168.2.1 255.255.255.0
No shutdown

Exit
end

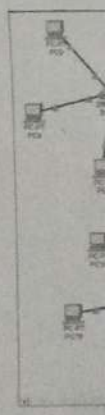
Step 4: Test communication between same subnet PC0, Ping 192.168.1.3
check PC1 responds.

Step 5: Test
subnets
cl

Step 6: T
* Dis
* A

Observation

Source
PC0
PC0
PC0



Result:

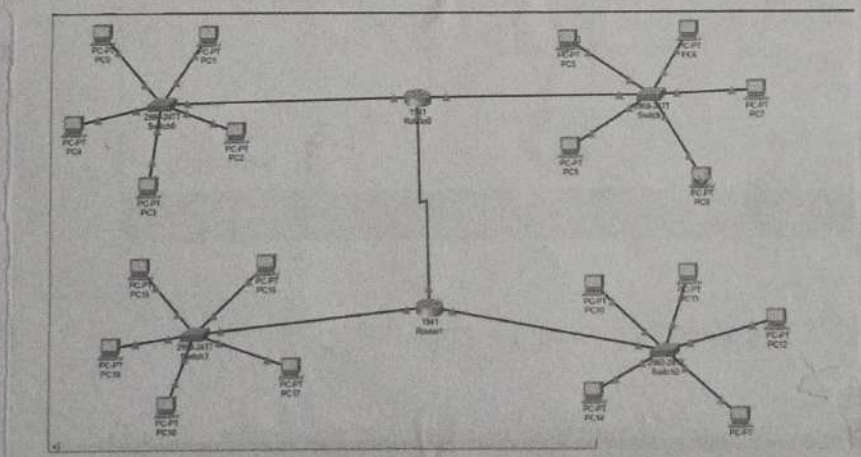
Successfu

Step5: Test communication between different subnets pc0, ping 192.168.2.2
check whether PC_2 responds

Step6: Test communication without Router
* Discount router links
* Attempt to ping PC_2 from pc0 again

Observation table

Source	Destination	Expected Result
pc0	PC_1	Successful
PC_0	PC_2	Successful when router is configured & connect
pc0	PC_3	Unsuccessful when the router connection between subnets is disabled



Result:
Thus the Subnetting was Implemented successfully using Cisco packet tracer

Exp-11 - Internetworking with Router in CISCO packet tracer
5/10/26 simulator.

10(a) Internetworking using Routers

Aim:
To design and configure simple Internetwork using router in CISCO packet tracer.

Algorithm:

* place router & two PCs in packet tracer
* connect PCs to router using straight through cable
* Assign IP address & subnet mask to PC & router interface
* configure router using CLI & enable them
* use ping to test connectivity.

Procedure:

* open packet tracer & create network topology
* configure router interface with IP address
* Assign IP address to the PC in different network
* enable router interface using no shutdown
* Test communication between PCs

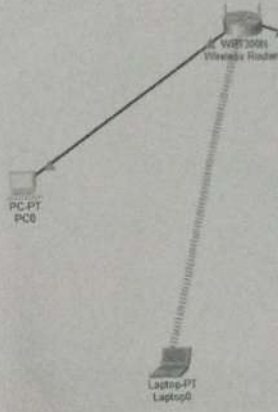
Result:
The Internetwork was configured successfully & connectivity between PCs was tested using router

10(b) wire

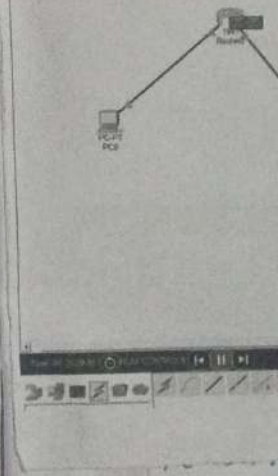
Aim:
To d
a wireles

Algorithm

* Place u



* E
* Co



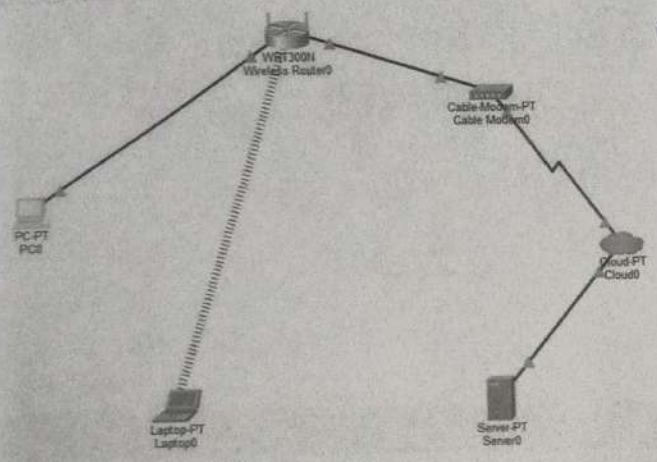
10 (b) wireless Router, DHCP Server & Internet cloud

Aim :

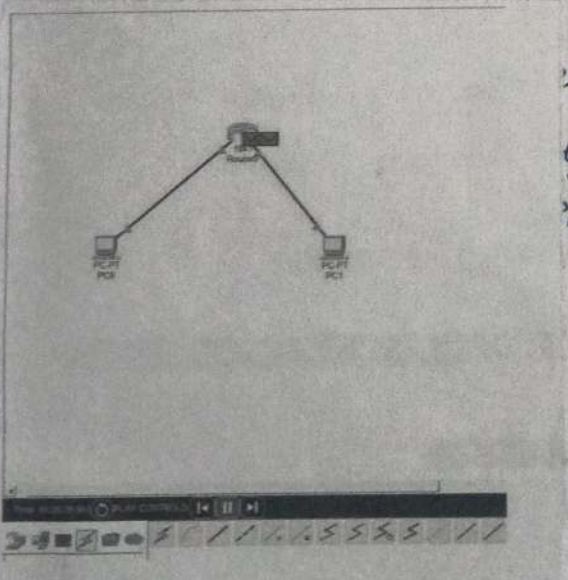
To design & configure and internetwork using a wireless router DHCP server & internet cloud

Algorithm:

* Place wireless router, server, PC & internet cloud
* network & DHCP service
* DNS setting
* IP address automatically
* PING command



* In packet tracer
* ~~Enable~~ DHCP & configure wireless router
* Configure server with IP & DNS setting
* wireless network



y using ping
ng ping cisco.com

The wireless internetwork was configured & DHCP network connectivity were tested

10(c) interconnect using switch, Router and Ethernet cables

Aim:

To design and configure an Internetwork using switch, router & Ethernet cable

Algorithm:

* place the router, switch & PCs in packet tracer
* connect PCs to switch & switch to router
* Assign IP address, subnet mask & default gateways
* configure & enable the router interfaces
* Test connectivity using PING

Procedure:

* create network topology
* connect devices using ethernet cables
* configure IP address
* enable router interface
* verify communication

Result:

The Internetwork was configured successfully. & connectivity between devices was verified

Exp. 12 Routing information protocol
13/10/26

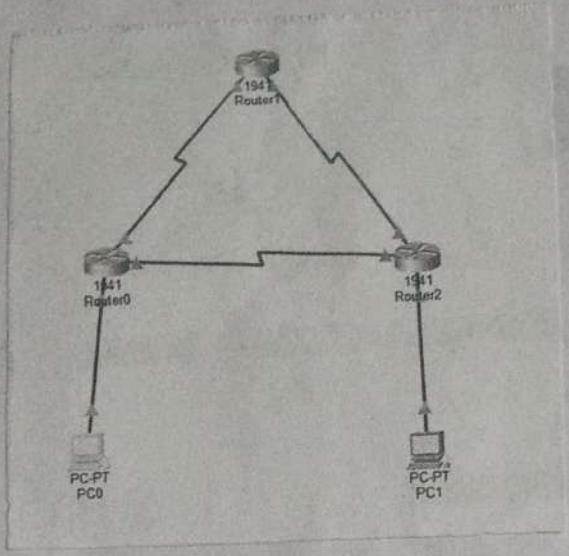
Aim:

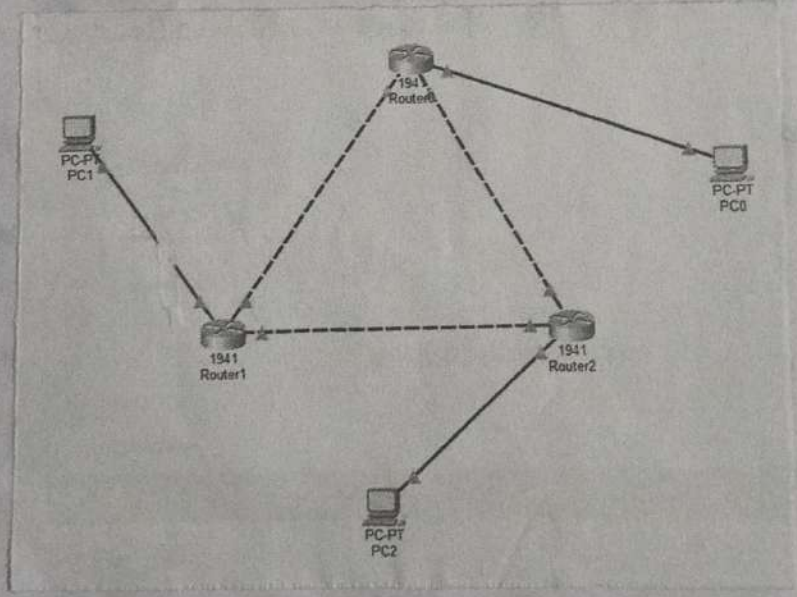
To simulate a static routing configuration using Cisco packet Tracer

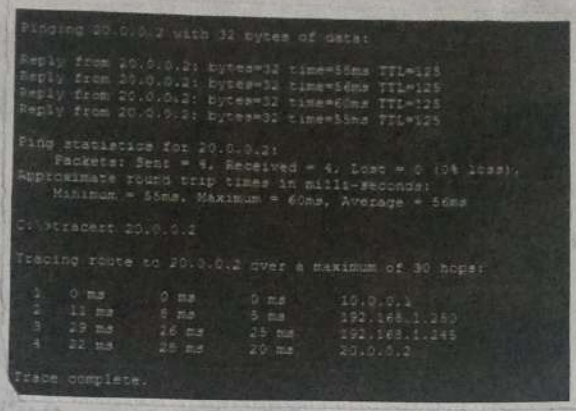
Procedure:

* Open cisco packet tracer and create the given network topology
* Connect a 3 routers and pc's using suitable cables
* Configure Ip address for routers and pc's
* Enable all router interface using no shut down
* Configure static routes on router 0, router 1 and router 2 using ip route.
* Set main and backup router using different administrative distance value
* Test connectivity
* use trace to verify the routing path.
* thus a static routing is successfully configured and verified




1941
Router1
1941
Router0
1941
Router2
PC-PT
PC0
PC-PT
PC1


1941
Router0
PC-PT
PC1
PC-PT
PC0
1941
Router1
1941
Router2
PC-PT
PC2



```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2   11 ms      6 ms      5 ms      192.168.1.250
  3   29 ms     26 ms     25 ms      192.168.1.245
  4   32 ms     28 ms     29 ms      20.0.0.2

Trace complete.
```

```
Router>enable
Router#show ip route static
S    30.0.0.0/8 [20/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S    30.0.0.0/8 [10/0] via 20.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2
```

Result:

Routing information protocol successfully configure and dynamic routing between PC0 and PC1 was verified.

Exp-13 Implement your own ping program
13/10/21

Aim:

To write a python program to implement the ping utility and check network connectivity between two hosts

Algorithm:

* Get destination IP address from user
* Send an ICMP echo request using the system ping command
* Receive the response from the destination
* Display whether host is reachable
* Stop the program

Code:

```
import subprocess
import platform
ip = input ("Enter destination IP address : ")
if platform.system().lower() == "windows":
    command = ["ping", "-n", "4", ip]
else:
    command = ["ping", "-c", "4", ip]
result = subprocess.run(command, capture_output=True,
                        text=True)
print (result.stdout)
if result.returncode == 0:
    print ("Host is reachable")
else:
    print ("Host is unreachable")
```

output :

I/P

Enter destination IP address

8.8.8.8

O/P

Reply form : 8.8.8.8

Host is reachable

Result:

Thus, the python program to check network connectivity using ping utility was executed successfully

Exp-14 Packet sniffing using Raw Sockets.
13/10/24

Aim:

To write python program using sockets to capture network packets and display their source IP destination IP and protocol

Algorithm:

* create raw socket
* capture incoming IP packets
* Extract IP header from each packet
* identify protocol used
* display packet information

code:

```
import socket
import struct

try:
    sniffer = socket.socket
        Socket.AF_INET,
        Socket.SOCK_RAW,
        Socket.IPPROTO_IP
except OSError as e:
    print("Socket creation failed:", e)
    Print("Run with administrator root privileges!")
    raise SystemExit
Print("Packet sniffer started. Press Ctrl+C to stop")
try:
    while True:
        packet, address = sniffer.recvfrom(65535)
    if len(packet) < 20:
        continue
```

```
version_ihl = packet[0]
version = version_ihl >> 4
ihl = (version_ihl & 0x0F) * 4
if version != 4 or len(packet) < ihl:
    continue
ttl = packet[8]
protocol = packet[9]
src_ip = socket.inet_ntoa(packet[12:16])
dst_ip = socket.inet_ntoa(packet[16:20])
protocols = {1: "ICMP",
             6: "TCP",
             17: "UDP"}
print("\n packet captured")
print("source ip:", src_ip)
print("Destination IP:", dst_ip)
print("Protocol:", protocols.get(protocol, str(protocol)))
print("TTL:", ttl)
print("packet size:", len(packet))
except keyboard Interrupt:
    print("\n packet sniffing stopped")
finally:
    sniffer.close()
```

Output:

Packet sniffer started.

Press ctrl + c to stop

Packet captured

Source IP : 192.168.1.10

Destination IP : 192.168.1.1

Protocol : TCP

TTL : 64

Packetsize : 60

Result:

Thus, the python program using raw sockets was implemented to capture network packets executed successfully.